

Swinburne University of Technology
School of Science, Computing and Engineering Technologies



Name: Chai Yen Thung

Student ID: 105025228

Unit Name: SWE40006 - Software Deployment and Evolution

Semester: Aug 2026

Lecturer: Dr. Robina Tinawin

Due Date: 4 September 2026

Submission Date: 4 September 2026

Project: Deployment Portfolio Task 1

Target Level Attempted:

High Distinction (Tasks 1.1, 1.2, 1.3, and 1.4)

Table of Contents

1.0	Declaration of Level Attempted.....	3
2.0	Task 1.1 - Pass Level	4
2.1	Environmental Setup.....	4
2.1.1	Problem.....	4
2.1.2	Resolution Strategy	4
2.2	Simple Desktop Application	5
2.3	WiX Packaging Configuration.....	6
2.4	Deployment Outcomes.....	7
3.0	Task 1.2 - Credit Level.....	9
3.1	Custom Desktop Application Development.....	9
3.2	WiX Packaging Configuration.....	10
3.3	Investigation and Resolution of Event Registration Errors	11
3.4	Deployment Outcomes.....	12
4.0	Task 1.3 – Distinction Level	14
4.1	Application Deployment with DLLs	14
4.1.1	MarkingRules.dll	14
4.1.2	ResultGenerator.dll	15
4.1.3	Desktop Application UI and C# Integration Code	16
4.2	WiX Packaging Configuration.....	18
4.3	Framework Retargeting Errors and Resolution	19
4.4	Deployment Outcomes.....	20
5.0	Task 1.4 - High Distinction.....	23
5.1	Window Application Packaging Project Setup	23
5.2	Package Installation and Troubleshooting	24
5.3	Detailed Microsoft Store Deployment Blueprint.....	26
5.4	Deployment Outcomes.....	28

1.0 Declaration of Level Attempted

I declare that I have completed all required steps up to and including **Task 1.4 for High Distinction**. This includes setting up WiX v4, building a custom application, structuring the application into a multi-library architecture with two external DLL dependencies and packaging the application into a signed MSIX package with a full Microsoft Store deployment blueprint.

[Link](#)

2.0 Task 1.1 - Pass Level

Task 1.1 was successfully completed by establishing a functional Windows Installer XML (WiX) deployment environment in Visual Studio, configuring a baseline “Hello World” simple desktop application, writing the required XML packaging configuration files and compiling a functional .msi installation package.

2.1 Environmental Setup

During the initial setup, a critical block occurred when I attempted to follow the provided walkthrough instructions using the course resources (wix40.exe).

2.1.1 Problem

Upon executing wix40.exe, the installation failed during the caching phase. The console and the installation logs displayed the following verification errors

```
[2C90:064C][2026-09-02T00:00:24]e000: Error 0x80091007: Hash mismatch for path: C:\ProgramData\Package Cache\unverified\Wix, expected: 800C00F91591D82892A1A31F235C48E8BAD1894F, actual: 16CC22C73B88298B538CDCF52BE3709231D61ECF  
[2C90:064C][2026-09-02T00:00:24]e000: Error 0x80091007: Failed to verify hash of payload: Wix
```

Figure 1: Compiler log displaying the package cache verification failure (Error 0x80091007 Hash Mismatch).

```
[223C:451C][2026-09-02T00:00:24]i399: Apply complete, result: 0x80091007, restart: None, ba requested restart: No
```

Figure 2: Setup logs confirming installation failure (Result: 0x80091007) when using the outdated bootstrapper.

I discovered the wix40.exe provided in the course resources was an early bootstrapper engine because it is an online bootstrapper, it attempts to fetch its primary payloads from the live WiX CDN servers at runtime. Overtime, the files hosted on the live servers have been updated, changing their SHA-1 hashes, because the legacy wix40.exe installer had the old preview hashes hardcoded into its security manifest, it detected a hash mismatch and automatically aborted the installation to protect the system. This made the wix40.exe installer completely unusable.

2.1.2 Resolution Strategy

To bypass the broken installer and still target the exact WiX v4 architecture required by the unit, I utilized the standard globally managed packaging tool via the .NET CLI. I opened the command prompt and ran `dotnet tool install --global wix --version 4.0.5`, which successfully pulled the stable version of the compiler.

```

C:\Users\user>dotnet tool install --global wix --version 4.0.5

Welcome to .NET 9.0!
-----
SDK Version: 9.0.317

Telemetry
-----
The .NET tools collect usage data in order to help us improve your experience. It is collected by Microsoft and shared with the community. You can opt-out of telemetry by setting the DOTNET_CLI_TELEMETRY_OPTOUT environment variable to '1' or 'true' using your favorite shell.

Read more about .NET CLI Tools telemetry: https://aka.ms/dotnet-cli-telemetry

-----
Installed an ASP.NET Core HTTPS development certificate.
To trust the certificate, run 'dotnet dev-certs https --trust'
Learn about HTTPS: https://aka.ms/dotnet-https

-----
Write your first app: https://aka.ms/dotnet-hello-world
Find out what's new: https://aka.ms/dotnet-whats-new
Explore documentation: https://aka.ms/dotnet-docs
Report issues and find source on GitHub: https://github.com/dotnet/core
Use 'dotnet --help' to see available commands or visit: https://aka.ms/dotnet-cli
-----
Tool 'wix' is already installed.

C:\Users\user>wix --version
4.0.5+b9b2f1b4

```

Figure 3: Verification of successful global WiX v4.0.5 installation via the .NET command line

Since WiX v4 was modernized into a standalone .NET command line tool, it no longer includes built in Visual Studio integration by default. To bridge this gap and allow the IDE to communicate with the core compiler, I installed the official HeatWave Visual Studio extension. This extension provided seamless MSBuild integration and registered the necessary WiX v4 project templates, allowing me to compile the installer directly from the Visual Studio interface.

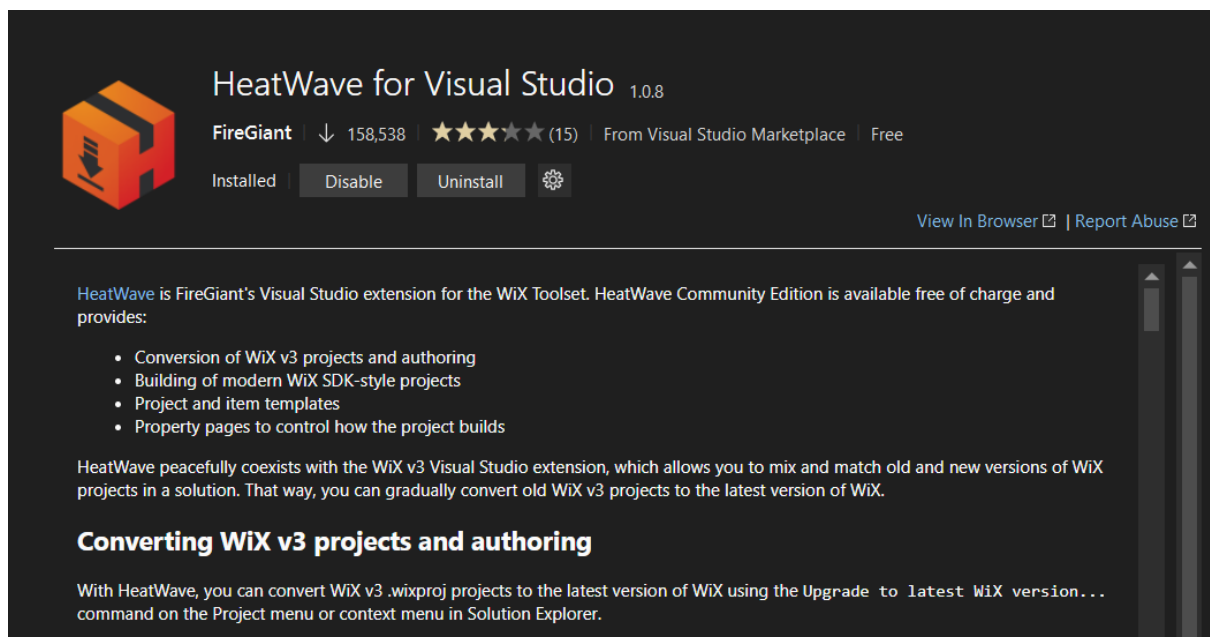


Figure 4: The HeatWave extension successfully installed and enabled inside Visual Studio.

2.2 Simple Desktop Application

A baseline Windows Forms desktop application named Task1.1 was created to verify the build engine, targeting the .NET Framework 4.6.2 as the walkthrough instructions. For this baseline application, a simple graphical user interface (GUI) was developed featuring a standard "Hello World!" text label. The project was built using the Release configuration, successfully compiling the execution payload to the particular directory.

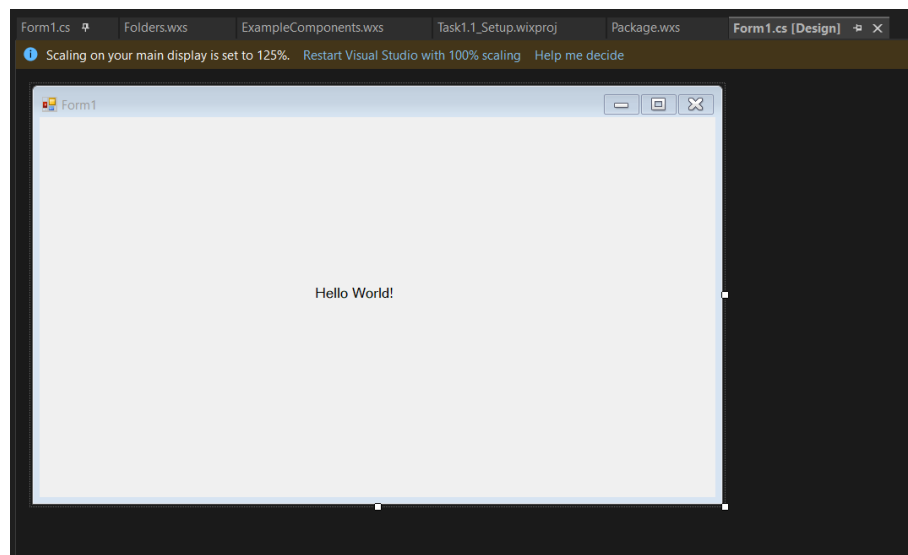


Figure 5: Screenshot shows simple Hello World desktop application

2.3 WiX Packaging Configuration

To deploy this application, the default XML template files generated by Visual Studio's WiX v4 project were modified. I have modified the main `<Package>` elements to define the installer's core metadata. I set the name to "Task1.1_Setup" and the Manufacturer to my name, while generating a unique Upgrade Code GUID to ensure the Windows Registry can handle future updates safely. Besides this, I also modified the `<MediaTemplate>` attribute to `EmbedCab="yes"` since this forces the compiler to pack all the installation files directly inside the single .msi file, making it much easier to distribute. Finally, I updated the `<File>` tag's `Source` attribute to point directly to my compiled C# execution payload located at `C:\Users\user\SWE40006\Task1.1\Task1.1\bin\Release\Task1.1.exe`, ensuring the WiX compiler grabs the correct Release build.

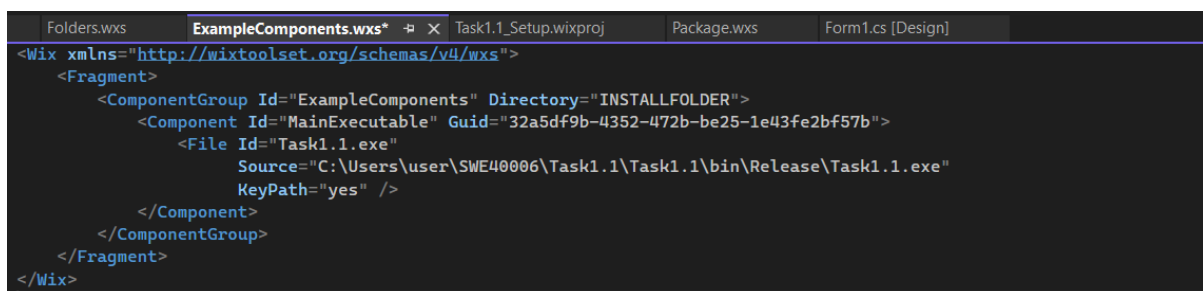


Figure 6: The ExampleComponents.wxs file configured to target your compiled Task1.1.exe file.

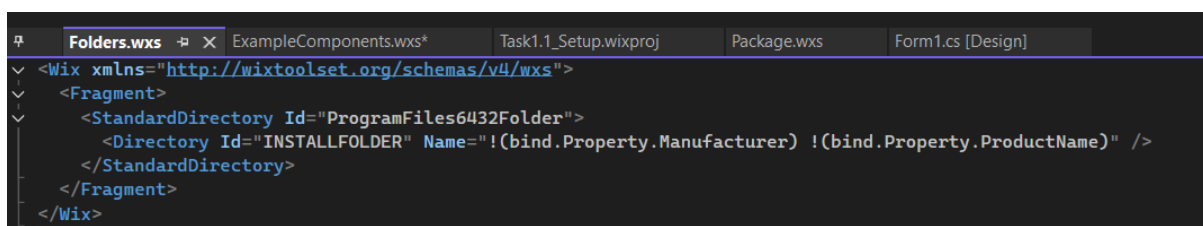


Figure 7: The Folders.wxs file defining the installation path in the system Program Files folder.

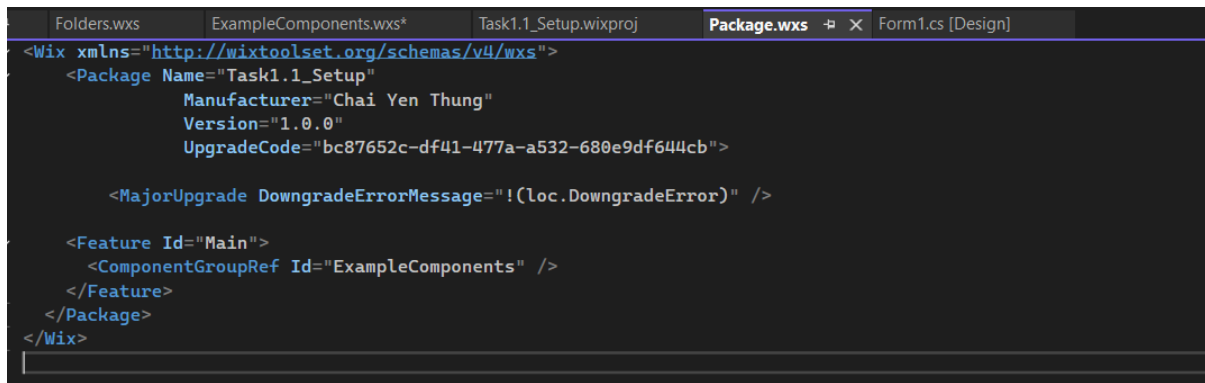


Figure 8: The Package.wxs file defining the installation path in the system Program Files folder.

2.4 Deployment Outcomes

To successfully verify the deployment pipeline, the process was executed and documented.

1. Baseline Application Verification

Before configuring the WiX installer package, the C# desktop application was compiled in the Release configuration to verify code syntax and output directory generation. The compiler successfully outputted the execution assembly (Task1.1.exe) to the local project build folder. Running the executable directly from this directory confirmed that the program initializes correctly, rendering the "Hello World!" graphical window.

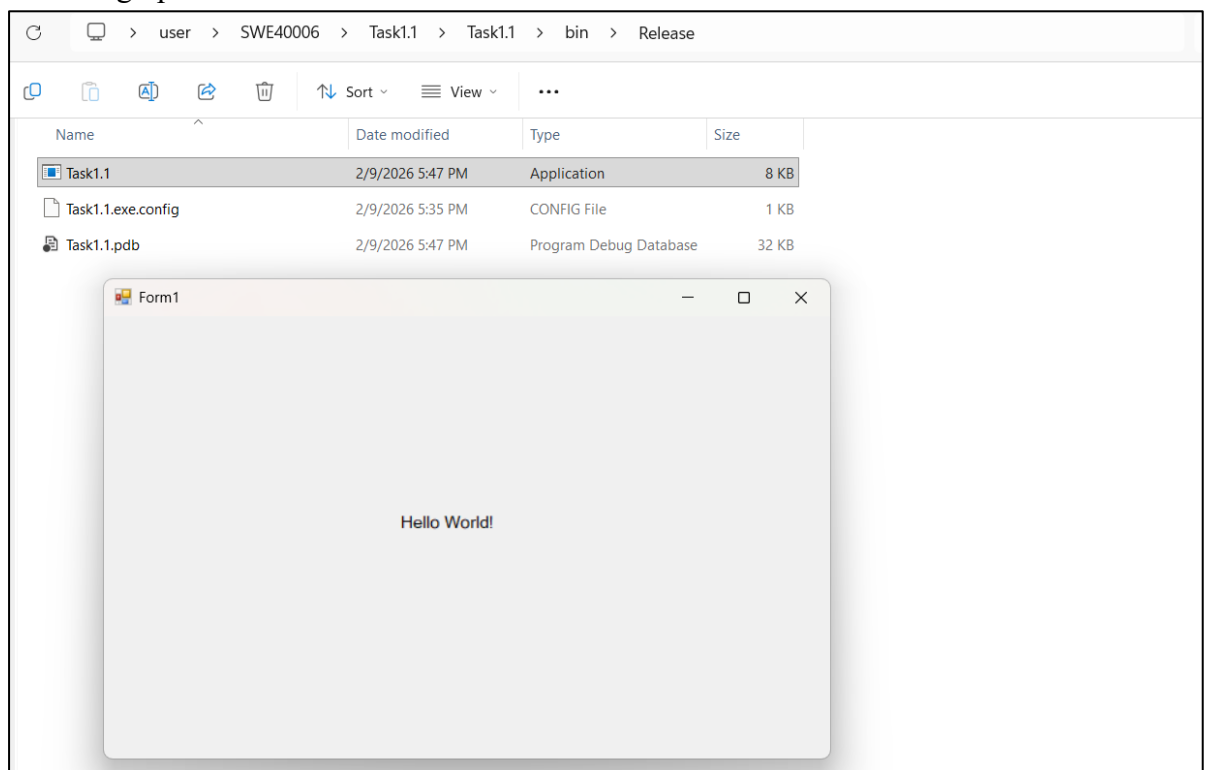
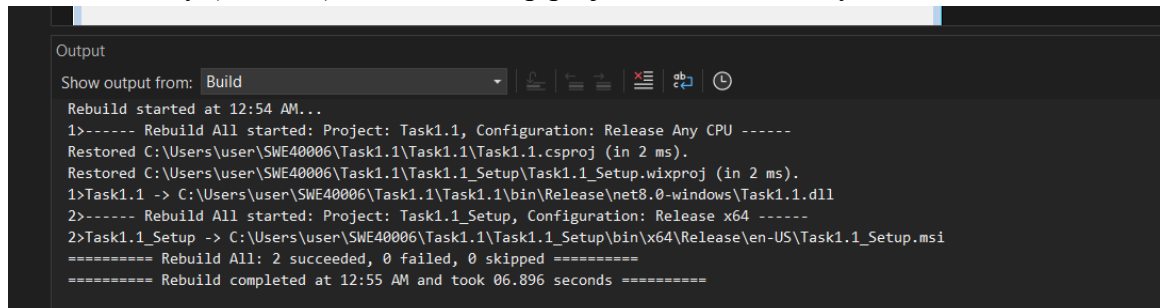


Figure 9: Screenshot of the compiled C# application output folder and the active "Hello World!" graphical interface executing successfully

2. WiX Installer Compilation

With the execution payload verified, the WiX Setup project was compiled in Visual Studio. The build completed with zero warnings and zero errors, confirming a clean compilation. Upon completion, the WiX compiler successfully outputted the physical installer binary (.msi file) inside the setup project's build directory.



```

Output
Show output from: Build
Rebuild started at 12:54 AM...
1>----- Rebuild All started: Project: Task1.1, Configuration: Release Any CPU -----
Restored C:\Users\user\SWE40006\Task1.1\Task1.1.csproj (in 2 ms).
Restored C:\Users\user\SWE40006\Task1.1\Task1.1_Setup\Task1.1_Setup.wixproj (in 2 ms).
1>Task1.1 -> C:\Users\user\SWE40006\Task1.1\bin\Release\net8.0-windows\Task1.1.dll
2>----- Rebuild All started: Project: Task1.1_Setup, Configuration: Release x64 -----
2>Task1.1_Setup -> C:\Users\user\SWE40006\Task1.1\Task1.1_Setup\bin\x64\Release\en-US\Task1.1_Setup.msi
===== Rebuild All: 2 succeeded, 0 failed, 0 skipped =====
===== Rebuild completed at 12:55 AM and took 06.896 seconds =====
  
```

Figure 10: Visual Studio console output verifying a successful rebuild with zero compilation errors.

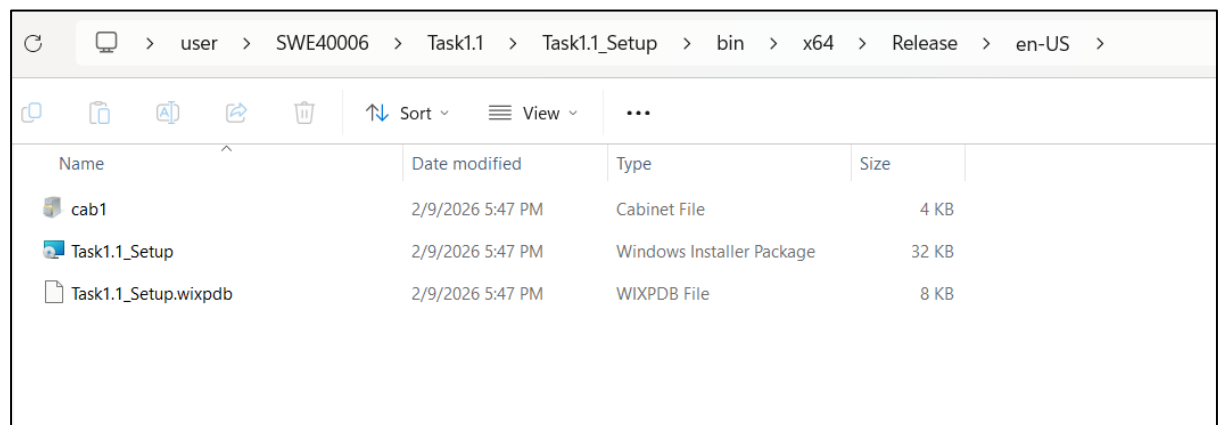


Figure 11: File Explorer displaying the successfully generated Task1.1_Setup.msi installer package

3. Deployment and System Registration

To deploy the application, the compiled .msi installer was executed on the local machine. To verify that the Windows operating system properly recognized and registered the package via the Windows Installer service, the system's "Installed Apps" panel was checked, confirming the app was officially installed.



Figure 12: Windows "Installed apps" settings panel confirming Task1.1_Setup is successfully registered on the system

3.0 Task 1.2 - Credit Level

The objective of Task 1.2 is to transition from a generic “Hello World” application to an original customized desktop application compiled in C# and deploy it successfully via the WiX packaging pipeline. For this task, a Square Calculator (x^2) application was designed and implemented.

3.1 Custom Desktop Application Development

To fulfill Task 1.2, a custom C# Windows Form application was developed to calculate the square of a user provided number. The interface layout is designed within Visual Studio with an instruction label, an input text box, a calculate button and a dynamic results output label.

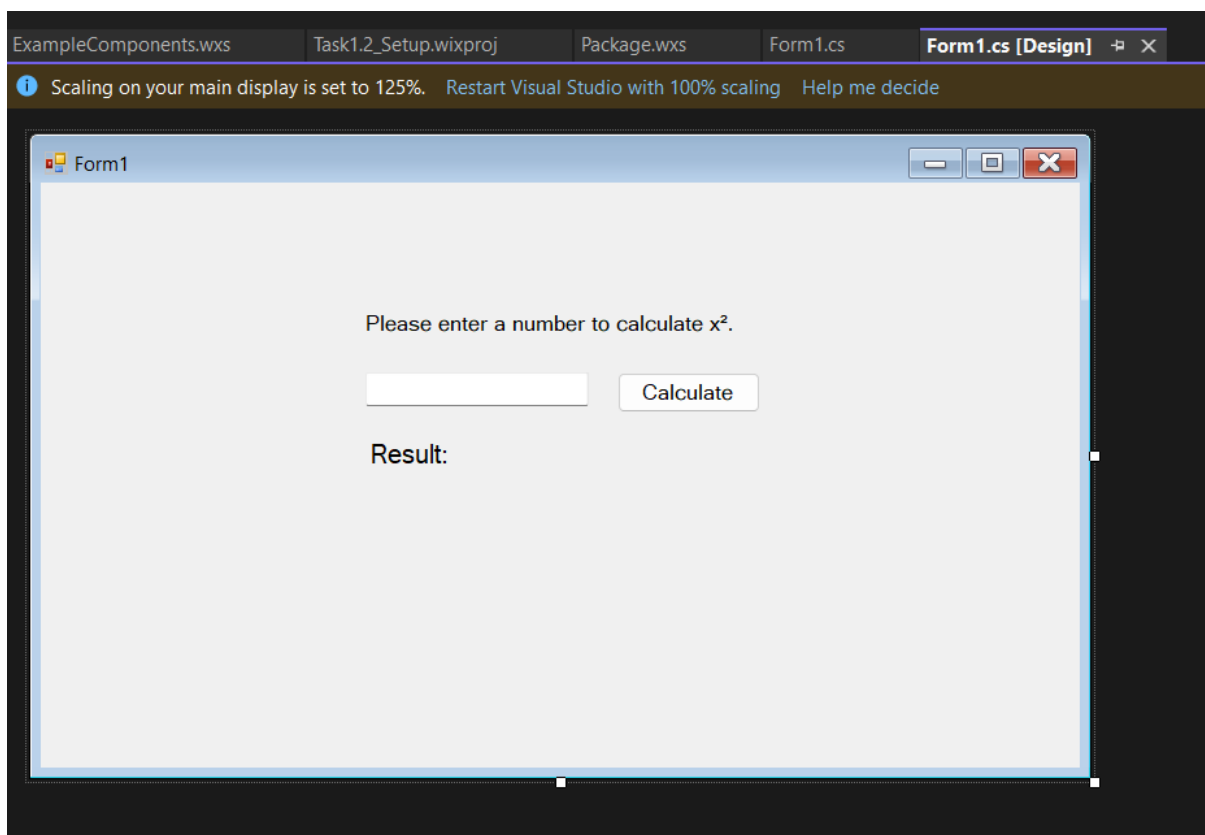


Figure 13: Visual Studio Form Designer window showing the custom Square Calculator

The logic behind the calculator is written directly within the form's class file, handling the input collection, mathematical operations and error handling routines.

```
namespace Task1._2
{
    3 references
    public partial class Form1 : Form
    {
        1 reference
        public Form1()
        {
            InitializeComponent();
        }

        1 reference
        private void btnCalculate_Click(object sender, EventArgs e)
        {
            string userInput = txtInput.Text;

            if (double.TryParse(userInput, out double number))
            {
                double result = number * number;

                lblResult.Text = $"Result: {result}";
            }
            else
            {
                MessageBox.Show("Please enter a valid number.", "Invalid Input");
            }
        }
    }
}
```

Figure 14: The custom C# code-behind file (Form1.cs) implementing the input validation and square calculation logic

The btnCalculate_Click event handler employs defensive programming to ensure application stability:

- **Safe Input Validation**
It extracts the user's string input and uses double.TryParse() rather than unsafe raw parsing. This prevents unhandled exceptions and application crashes if non-numeric characters are entered.
- **Computation & Output**
Validated numbers are squared (number * number), and the result is dynamically injected into the UI label using C# String Interpolation (\$"Result: {result}").
- **Error Feedback**
Invalid inputs trigger the else block, launching a user-friendly MessageBox prompt instead of crashing, ensuring a robust execution loop.

3.2 WiX Packaging Configuration

Since the baseline deployment environment was successfully established in Task1.1, the WiX packaging configuration for Task1.2 followed a similar structure. However, critical metadata was updated to "Task1.2_Setup" and brand new unique GUIDs were generated for both the Upgrade Code and the <Component> ID to prevent conflicts with previous installations in the Windows Registry. Finally, the <File> Source path was rerouted to target the new compiled payload.

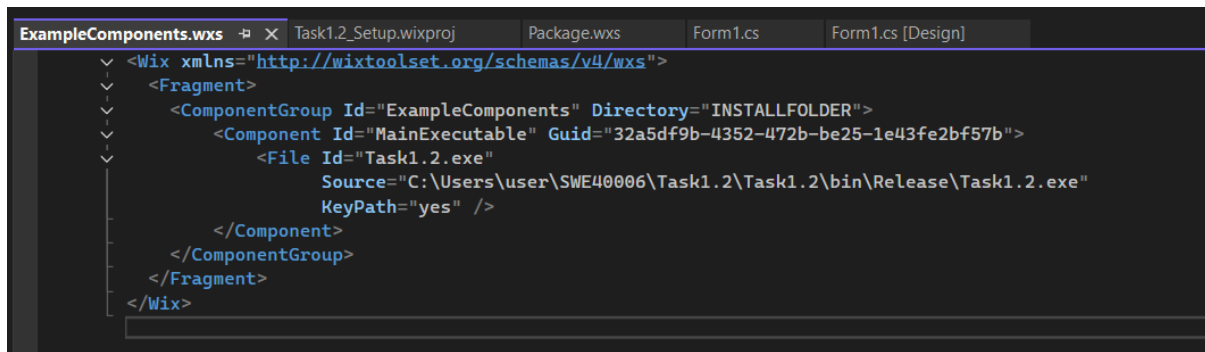


Figure 15: The ExampleComponents.wxs file mapped to compile the custom Task1.2.exe application executable

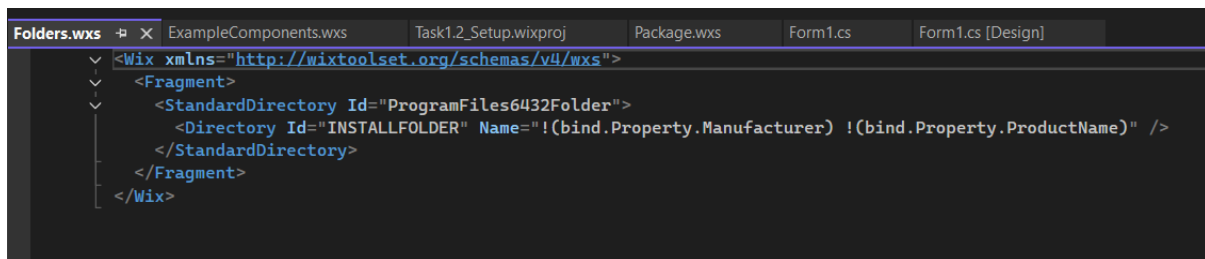


Figure 16: The Folders.wxs file specifying the target installation directory layout for the custom application package

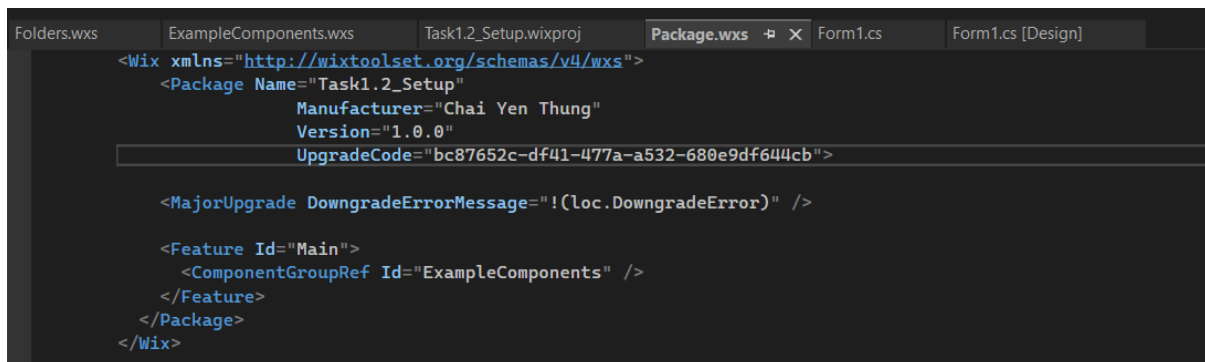


Figure 17: The Package.wxs file defining the setup metadata, manufacturer identity, and upgrade rules for Task1.2_Setup

3.3 Investigation and Resolution of Event Registration Errors

During the initial execution of the custom desktop application, a critical functionality issue arose where clicking the btnCalculate button had no effect.

- **Problem**

Although the code-behind file contained the correct logic inside the btnCalculate_Click method, the button did not trigger the calculation. Investigation of the background designer file (Form1.Designer.cs) revealed that the button control had no registered subscriber to its Click event. Writing the method in code-behind is not sufficient; Windows Forms requires explicit event-binding registration in the designer.

- **Resolution**

The event was bound using Visual Studio's Properties window. The button control was selected, the view was switched to the Events tab, and the Click event dropdown was configured to target `btnCalculate_Click`. This generated the necessary registration line in the designer file, linking the UI element directly to the code logic.

3.4 Deployment Outcomes

To verify the successful deployment of the custom calculator application, the process was executed and documented.

1. Custom Application Local Execution

Before compiling the installer package, the custom C# Windows Forms application was built in the Release configuration. The compiled execution files were successfully outputted to the local directory. Running the executable directly from this folder confirmed that the program initializes correctly and performs mathematical operations. Typing 9 and clicking Calculate button successfully evaluated and rendered Result: 81 on the screen.

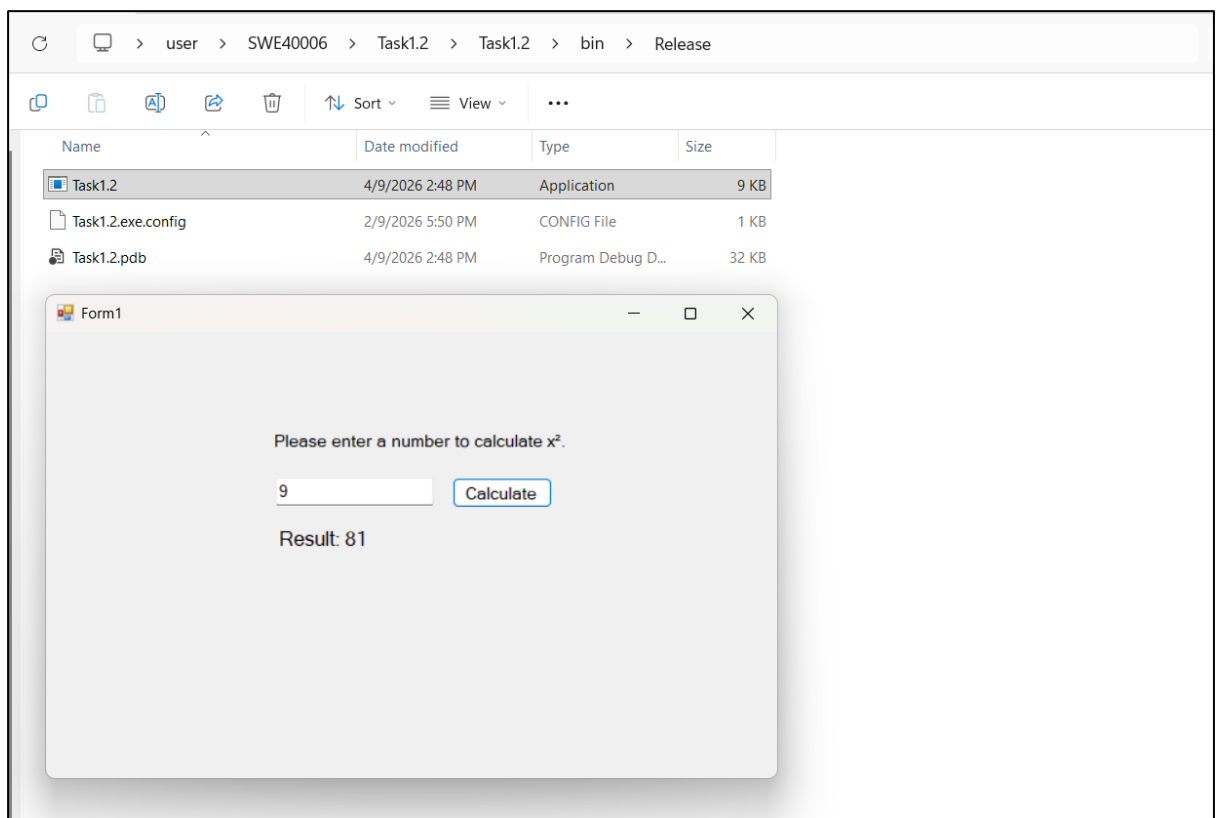


Figure 18: Screenshot of the release output folder and the active calculator interface.

2. WiX Installer Compilation

With the executable and application assets verified, the WiX Setup project (Task1.2_Setup) was opened in Visual Studio. The build configuration was set to Release (x64) and compiled. The Visual Studio output console confirmed that the build finished successfully with zero errors. Upon completion, the WiX compiler successfully outputted the physical Windows Installer Package (Task1.2_Setup.msi) to the target build directory.

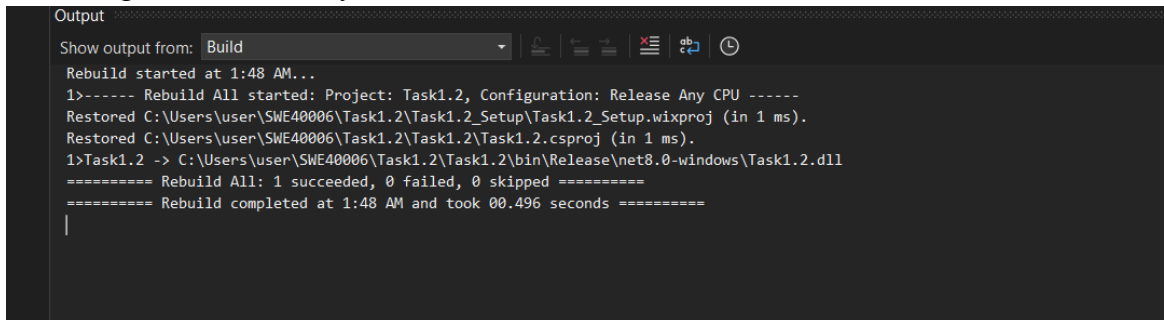


Figure 19: Screenshot of the Visual Studio IDE displaying zero build errors.

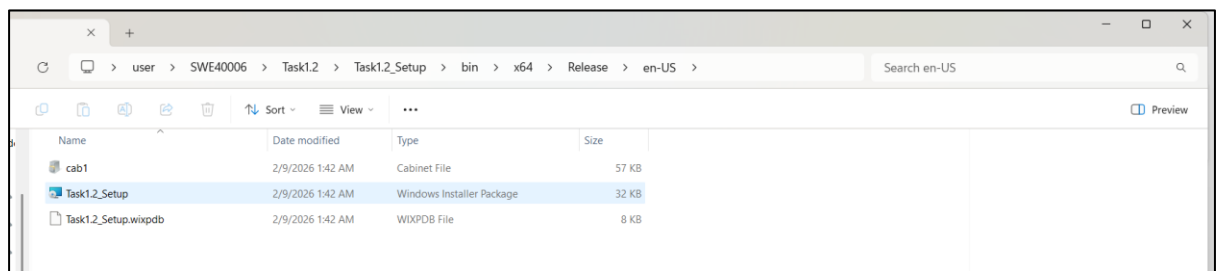


Figure 20: Screenshot of Windows File Explorer confirming the successful generation of the Task1.2_Setup.msi installer package.

3. Deployment and System Registration

To verify that the Windows operating system successfully registered the deployment, the system's "Installed apps" settings panel was opened. The search results confirmed that Task1.2_Setup was registered.

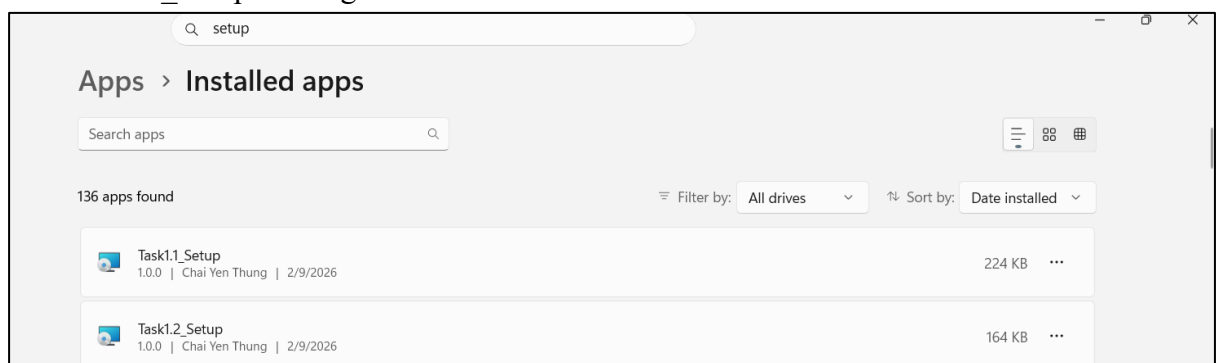


Figure 21: Windows "Installed apps" settings panel confirming Task1.2_Setup is successfully registered on the system

4.0 Task 1.3 – Distinction Level

Task 1.3 transitions the application from a single executable into a modern and decoupled multi-project architecture. This demonstrates the ability to package and deploy multiple dynamic link libraries dependencies using a single WiX v4 deployment script. To achieve this, this core calculation and formatting logic of the Grade Calculator were abstracted into two separate class libraries which is MarkingRules.dll and ResultGenerator.dll.

4.1 Application Deployment with DLLs

The application was designed around a multi-project modular architecture to satisfy the requirements of Task1.3. The system separated into one main presentation layer (Task1.3.exe) and two independent and custom-compiled dynamic link libraries (MarkingRules.dll and ResultGenerator.dll).

4.1.1 MarkingRules.dll

The MarkingRules library implements the core academic grading logic of the application, separating the calculation and classification rules from the presentation layer. The CalculateWeightedAverage method processes the student's grades and their corresponding assessment weights to calculate a proportional weighted average. This approach ensures that each assessment contributes to the result according to its assigned weighting, even when the total weight does not equal exactly 100%. The method also includes a validation check for a zero total weight, preventing a division-by-zero error. Following the calculation, the GetGradeBand method evaluates the resulting score against predefined academic boundaries and assigns the appropriate classification, from Fail through to High Distinction.

```
1 reference
public static double CalculateWeightedAverage(double[] grades, double[] weights)
{
    double weightedSum = 0;
    double totalWeight = 0;

    for (int i = 0; i < grades.Length; i++)
    {
        weightedSum += grades[i] * weights[i];
        totalWeight += weights[i];
    }

    if (totalWeight == 0) return 0;

    return weightedSum / totalWeight;
}

1 reference
public static string GetGradeBand(double score)
{
    if (score >= 80) return "High Distinction (HD)";
    if (score >= 70) return "Distinction (D)";
    if (score >= 60) return "Credit (C)";
    if (score >= 50) return "Pass (P)";
    return "Fail (F)";
}
```

Figure 22: The custom math library source code (Evaluator.cs) implementing the weighted average calculation

4.1.2 ResultGenerator.dll

The ResultGenerator.dll library is responsible for generating and locally archiving the application's final academic result report. The GenerateDetailedReport method combines the assignment names, grades, weights, final weighted score, and grade classification into a structured text-based report. A 'StringBuilder' is used to efficiently construct the multiline output, while string.Format() applies fixed-width alignment to present the assignment information in a clear tabular format. The method also filters out empty assignment names so that only valid entries are included in the generated report.

In addition, the report records the current date and time using DateTime.Now which providing a timestamp for when the result was generated. After the report is constructed, Environment.GetFolderPath() identifies the user's standard Documents directory, while Path.Combine() creates the file path in a platform-safe manner. File.WriteAllText() then saves the generated report as 'Weighted_Grade_Report.txt', providing persistent local storage of the result.

```
1 reference
public static string GenerateDetailedReport(string[] names, double[] grades, double[] weights, double finalScore, string finalGrade)
{
    string timeStamp = DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss");

    StringBuilder sb = new StringBuilder();
    sb.AppendLine("=====");
    sb.AppendLine("                RESULT REPORT                ");
    sb.AppendLine("=====");
    sb.AppendLine($"Generated on: {timeStamp}");
    sb.AppendLine("-----");
    sb.AppendLine(string.Format("{0,-15} | {1,-10} | {2,-10}", "Assignment", "Grade (%)", "Weight (%)"));
    sb.AppendLine("-----");

    for (int i = 0; i < names.Length; i++)
    {
        if (!string.IsNullOrEmpty(names[i]))
        {
            sb.AppendLine(string.Format("{0,-15} | {1,-10}% | {2,-10}%", names[i], grades[i], weights[i]));
        }
    }

    sb.AppendLine("-----");
    sb.AppendLine($"Final Weighted Score: {Math.Round(finalScore, 2)}%");
    sb.AppendLine($"Grade Achieved: {finalGrade}");
    sb.AppendLine("=====");

    string report = sb.ToString();

    try
    {
        string documentsPath = Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments);
        // Saves the report under a generic report name on disk
        string fileName = Path.Combine(documentsPath, "Weighted_Grade_Report.txt");
        File.WriteAllText(fileName, report);
    }
    catch (Exception) { }

    return report;
}
```

Figure 23: The report formatting library source code (Generator.cs) handling text layout compilation and file saving

4.1.3 Desktop Application UI and C# Integration Code

The btnCalculate button controls the main calculation workflow of the application. When clicked, it collects all the users input from the five input rows and processes each row using a loop. Empty rows are skipped which allow users to enter only the assessments they require. The input values are validated using `double.TryParse()` to ensure that grades and weights contain valid numerical values. Additional boundary checks ensure that grades remain within 0–100% and that weights are not negative. If invalid input is detected, an error message is displayed, and the calculation is stopped to prevent incorrect results. After validation, the collected data is converted into arrays and passed to `MarkingRules.dll`. These results are then passed to `ResultGenerator.dll` to generate and save the formatted result report. Finally, the completed report is displayed to the user through a Message Box that provide immediate feedback within the GUI.

Besides, the btnReset button provides a simple way to clear the current user input and return the form to its initial state. When clicked, all 15 textboxes are grouped into an array. A 'foreach' loop then iterates through each textbox and calls the `Clear()` method to remove its contents. This allows users to quickly discard the existing entries and perform a new calculation without manually deleting each field.

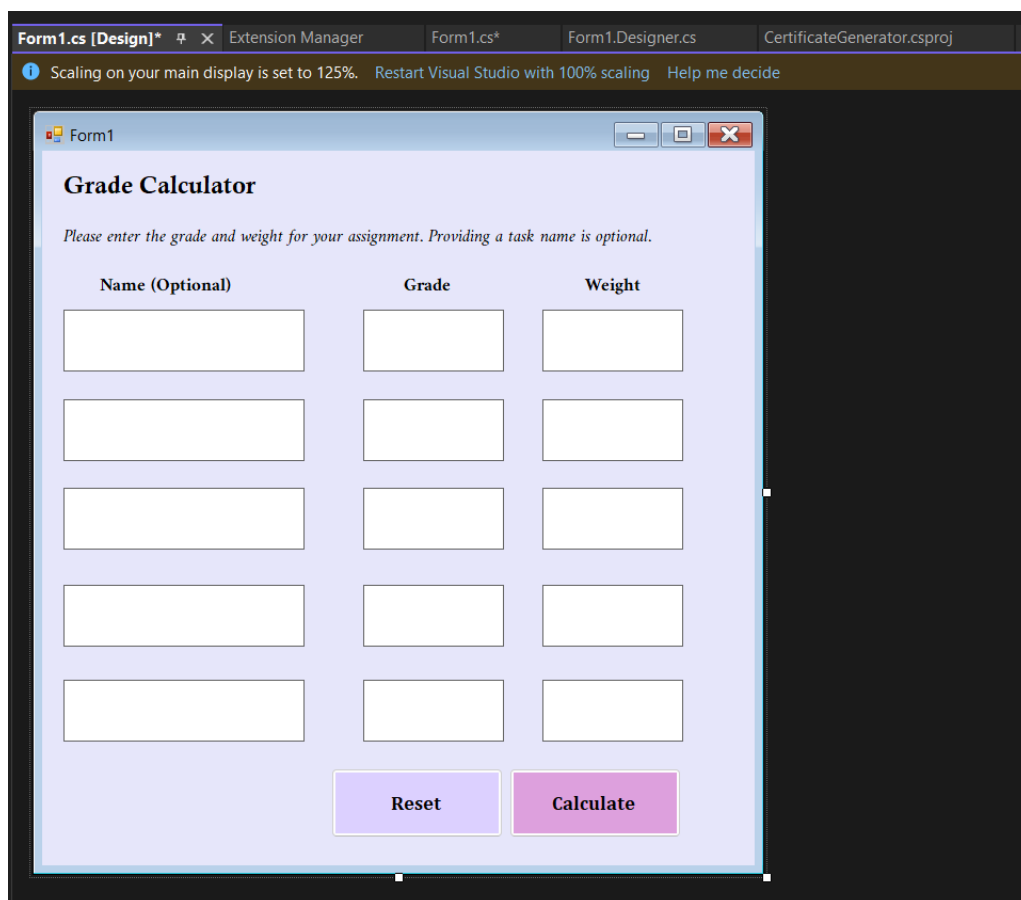


Figure 24: Form Designer window displaying the visual layout of the Grade Calculator UI


```
// Group 5 rows of TextBoxes into arrays
TextBox[] asgNames = { txtInput1, txtInput2, txtInput3, txtInput4, txtInput5 };
TextBox[] asgGrades = { txtGrade1, txtGrade2, txtGrade3, txtGrade4, txtGrade5 };
TextBox[] asgWeights = { txtWeight1, txtWeight2, txtWeight3, txtWeight4, txtWeight5 };

List<string> activeNames = new List<string>();
List<double> activeGrades = new List<double>();
List<double> activeWeights = new List<double>();

// Loop through all 5 rows
for (int i = 0; i < 5; i++)
{
    string name = asgNames[i].Text.Trim();
    string gradeText = asgGrades[i].Text.Trim();
    string weightText = asgWeights[i].Text.Trim();

    if (string.IsNullOrEmpty(name) && string.IsNullOrEmpty(gradeText) && string.IsNullOrEmpty(weightText))
    {
        continue;
    }

    double grade = 0;
    double weight = 0;

    if (!string.IsNullOrEmpty(gradeText) && !double.TryParse(gradeText, out grade))
    {
        MessageBox.Show($"Please enter a valid numeric grade in row {i + 1}. \n(Found invalid value: '{gradeText}')" , "Input Error");
        return;
    }

    if (!string.IsNullOrEmpty(weightText) && !double.TryParse(weightText, out weight))
    {
        MessageBox.Show($"Please enter a valid numeric weight in row {i + 1}. \n(Found invalid value: '{weightText}')" , "Input Error");
        return;
    }
}
```

Figure 25: Form1.cs button Calculate click handler code

```
// Value boundaries validation
if (grade < 0 || grade > 100 || weight < 0)
{
    MessageBox.Show($"Grade in row {i + 1} must be between 0-100%, and weight cannot be negative.", "Input Error");
    return;
}

activeNames.Add(string.IsNullOrEmpty(name) ? $"Task {i + 1}" : name);
activeGrades.Add(grade);
activeWeights.Add(weight);
}

// Ensure at least one active row has been filled out
if (activeWeights.Count == 0)
{
    MessageBox.Show("Please fill out at least one row with a valid grade and weight.", "Input Error");
    return;
}

string[] finalNames = activeNames.ToArray();
double[] finalGrades = activeGrades.ToArray();
double[] finalWeights = activeWeights.ToArray();

// Calculate the weighted average using MarkingRules.dll
double finalScore = Evaluator.CalculateWeightedAverage(finalGrades, finalWeights);
string finalGrade = Evaluator.GetGradeBand(finalScore);

// Generate the formatted transcript report using ResultGenerator.dll
string detailedReport = Generator.GenerateDetailedReport(finalNames, finalGrades, finalWeights, finalScore, finalGrade);

//Display in a clean pop-up window (MessageBox)
MessageBox.Show(detailedReport, "Grade Report", MessageBoxButtons.OK, MessageBoxIcon.Information);
}
```

Figure 26: Form1.cs button Calculate click handler code

```
1 reference
private void btnReset_Click(object sender, EventArgs e)
{
    // Gather all textboxes in the grid
    TextBox[] allTextBoxes = {
        txtInput1, txtInput2, txtInput3, txtInput4, txtInput5,
        txtGrade1, txtGrade2, txtGrade3, txtGrade4, txtGrade5,
        txtWeight1, txtWeight2, txtWeight3, txtWeight4, txtWeight5
    };

    // Loop and clear the text inside every box
    foreach (TextBox tb in allTextBoxes)
    {
        tb.Clear();
    }
}
```

Figure 27: Form1.cs button Reset click handler code

4.2 WiX Packaging Configuration

The WiX v4 configuration was updated to package the complete multi project application including the main executable and its two external DLL dependencies. The ExampleComponents component group places all required files into the INSTALLFOLDER which ensure that the application has access to its supporting libraries after installation. Three separate components are defined: “MainExecutable” packages “Task1.3.exe”, while “MarkingRulesDLL” and “ResultGeneratorDLL” package the two external class libraries required by the application. Besides, the package definition was also customised to identify the application as Task1.3_Setup, with my name specified as the manufacturer and version 1.0.0 assigned to the release.

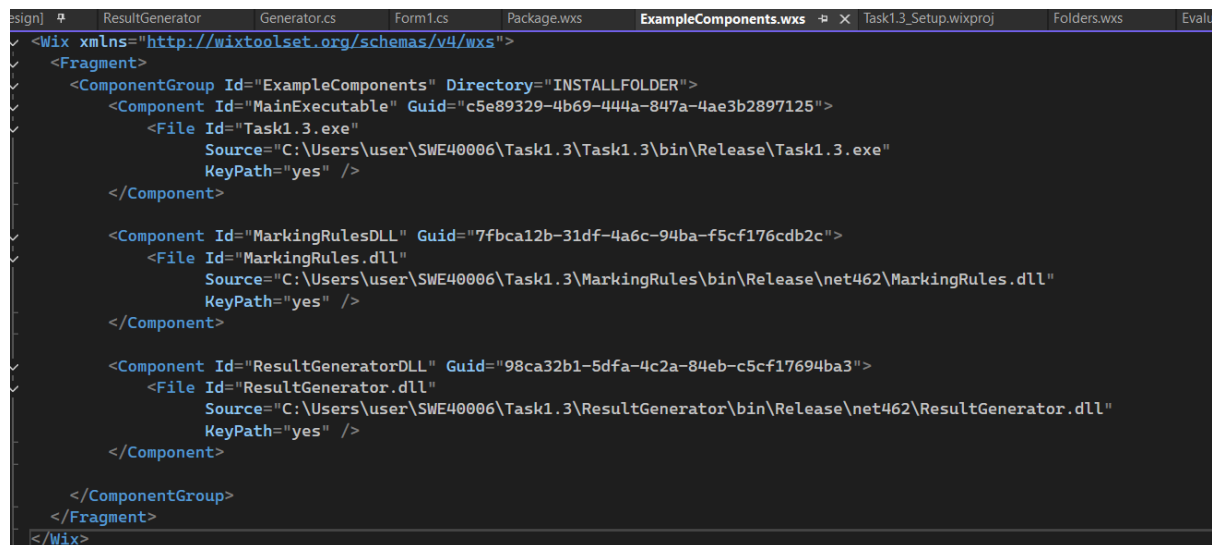


Figure 28: The ExampleComponents.wxs file configured to package the executable and both DLL dependencies.

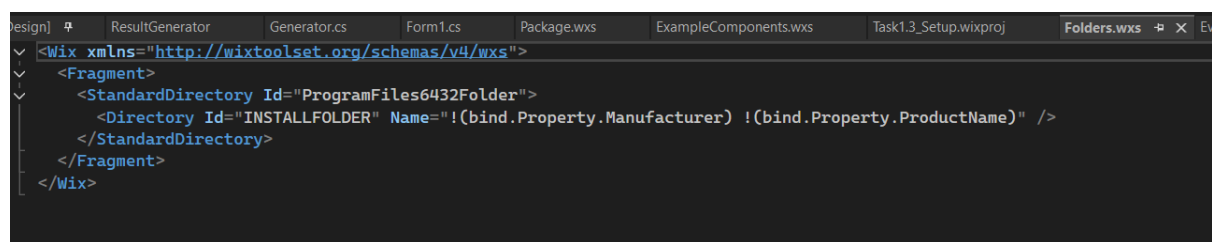


Figure 29: The Folders.wxs file defining the target installation folder layout in Program Files

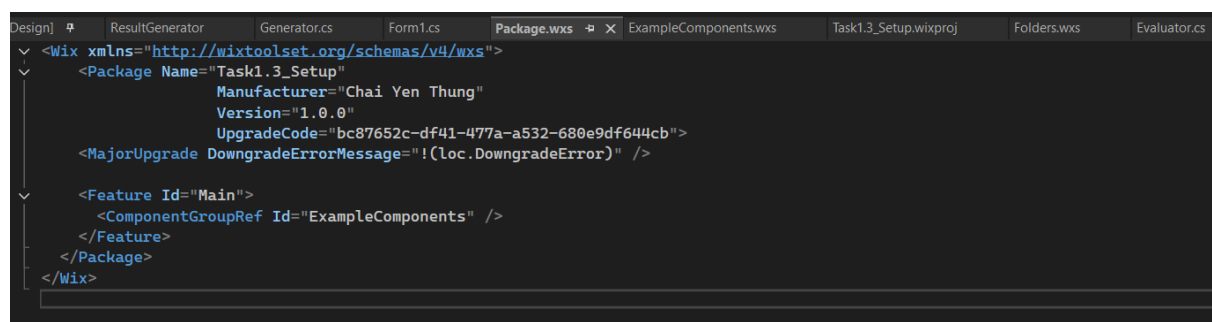


Figure 30: The Package.wxs file defining the setup metadata, manufacturer, and upgrade rules for Task1.3_Setup

4.3 Framework Retargeting Errors and Resolution

During the integration of the external class libraries, several compiler errors were encountered.

1. Symptoms

The first error identified a framework incompatibility between the main application and the external library. Besides, there is also a second error was related to the C# language configuration: `Invalid 'nullable' value: 'Enable' for C# 7.3. Please use language version '8.0' or greater.`

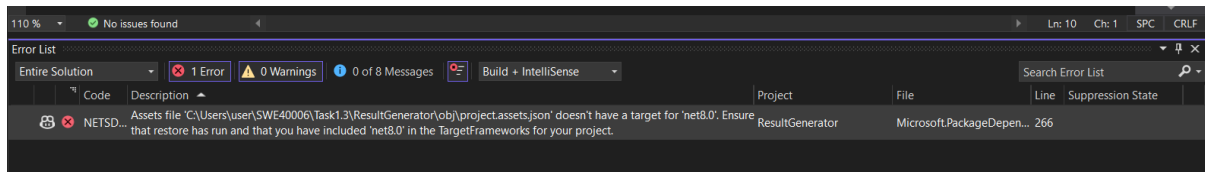


Figure 31: MSBuild compiler displaying the dependency target framework mismatch error in the Error List

2. Technical Root Cause

The errors were caused by differences between the framework and project configurations used by the application and the newly created class libraries. The main Windows Forms application targets the legacy .NET Framework 4.6.2, while the class libraries were initially created using a modern Visual Studio template targeting .NET 8.0. Because these frameworks use different runtime architectures, the .NET Framework application could not directly reference the .NET 8.0 class libraries.

In addition, the newer class-library templates included modern C# project settings such as `<Nullable>enable</Nullable>` and `<ImplicitUsings>enable</ImplicitUsings>`. These settings are not compatible with the older C# language configuration used by the .NET Framework project, resulting in the second compiler error.

3. Resolution

The compatibility issues were resolved by directly editing the .csproj configuration files of the external libraries. Both library projects were retargeted from net8.0 to net462, aligning them with the framework used by the main Windows Forms application. The unsupported `<Nullable>` and `<ImplicitUsings>` configuration elements were also removed from the project files.

After these changes, the application and both external libraries were able to target the same framework, allowing the DLL projects to be successfully referenced and compiled as part of the solution. This troubleshooting process demonstrated the importance of maintaining framework compatibility when integrating separately developed components.

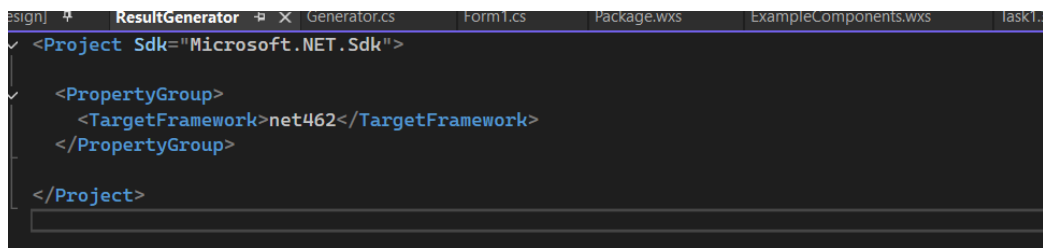


Figure 32: The edited ResultGenerator.csproj project file successfully retargeted to target net462

4.4 Deployment Outcomes

To verify the successful deployment of this multi-dependency project, the following sequential verification steps were performed.

1. Solution Rebuild in Visual Studio

The entire solution was compiled using the Release configuration. The Visual Studio output console confirmed that all three projects rebuilt successfully with zero errors.

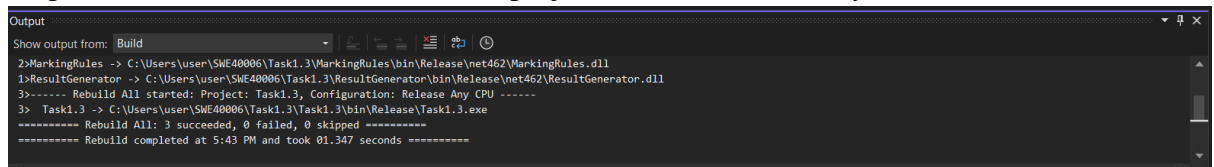


Figure 33: Screenshot of the Visual Studio output panel displaying a successful compilation of the three projects in the solution with zero build errors.

2. Verification of Independent DLL Compilation

To ensure that each class library compiled its respective logic successfully, the individual output directories were inspected. Both projects successfully compiled their source files into their targeted .net462 release directories.

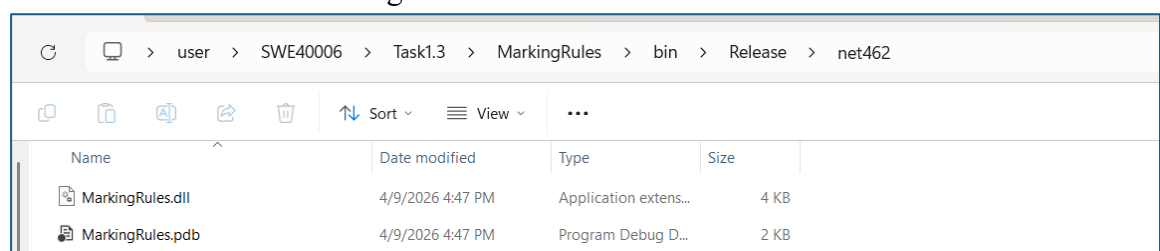


Figure 34: Screenshot of the MarkingRules compilation folder confirming MarkingRules.dll has been successfully generated

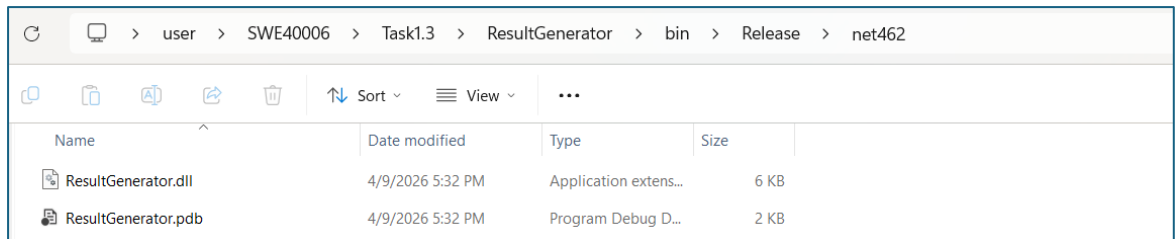


Figure 35: Screenshot of the ResultGenerator compilation folder confirming ResultGenerator.dll has been successfully generated

3. Verification of Main Application Assembly Folder

This folder confirms that the execution payload Task1.3.exe sits side-by-side with its two required dependencies, MarkingRules.dll and ResultGenerator.dll.

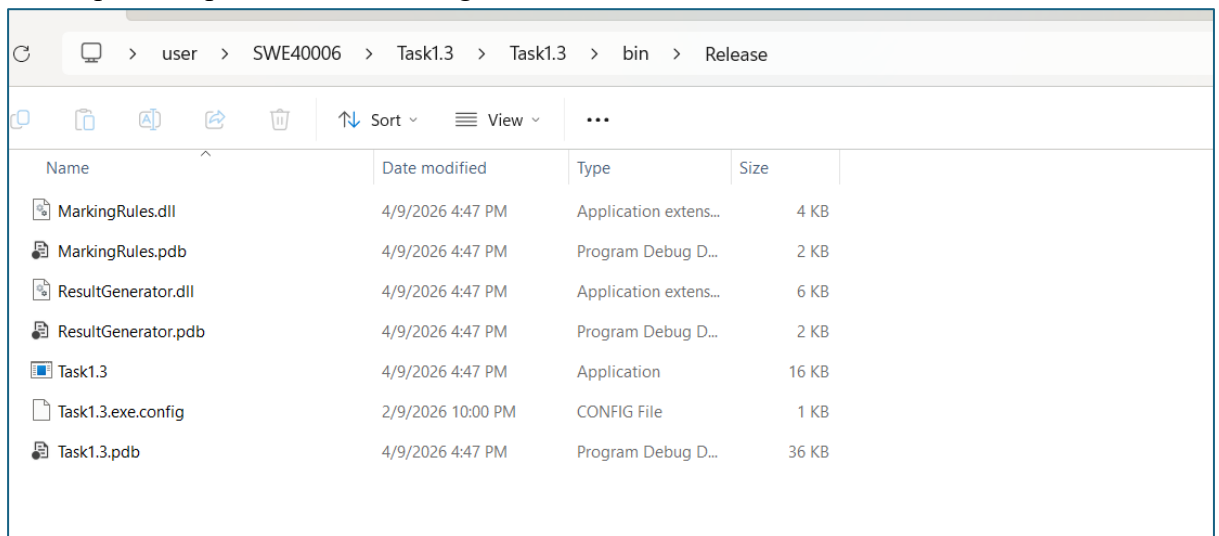


Figure 36: Screenshot of the main desktop project's Release folder, verifying the executable has successfully resolved and copied both dynamic library dependencies locally

4. WiX Setup Compilation

The WiX Setup project (Task1.3_Setup) was built in the Release configuration. The build completed successfully and outputted the installation package Task1.3_Setup.msi to the targeted folder.

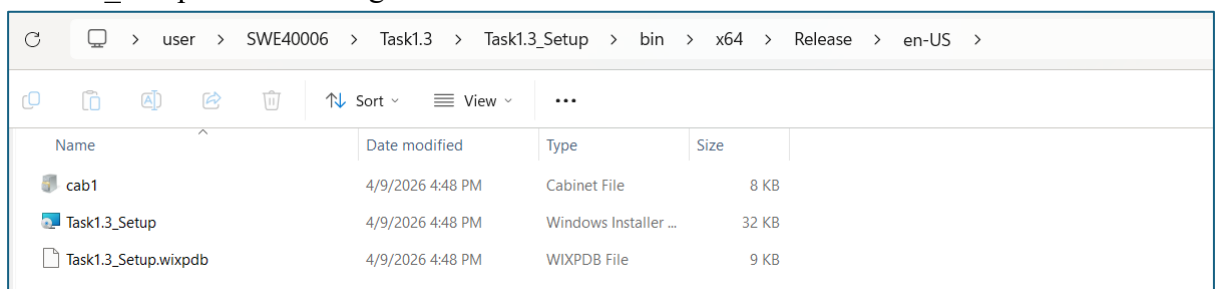


Figure 37: Screenshot of the en-US release directory, verifying the successful generation of the Task1.3_Setup.msi installer package.

5. System Registration

The newly compiled Task1.3_Setup.msi file was launched to install the updated system on the machine. To verify that the Windows Installer service registered the package correctly, the "Installed apps" settings panel was opened.

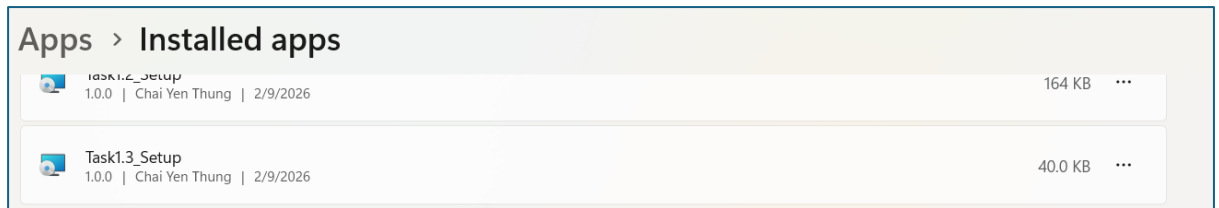


Figure 38: Screenshot of the Windows "Installed apps" settings panel confirming Task1.3_Setup is registered as an installed software component on the system

6. Verification of Application Execution & Multi-DLL Integration

The application was launched and verify that the installed executable successfully resolves and loads both DLL assemblies at runtime.

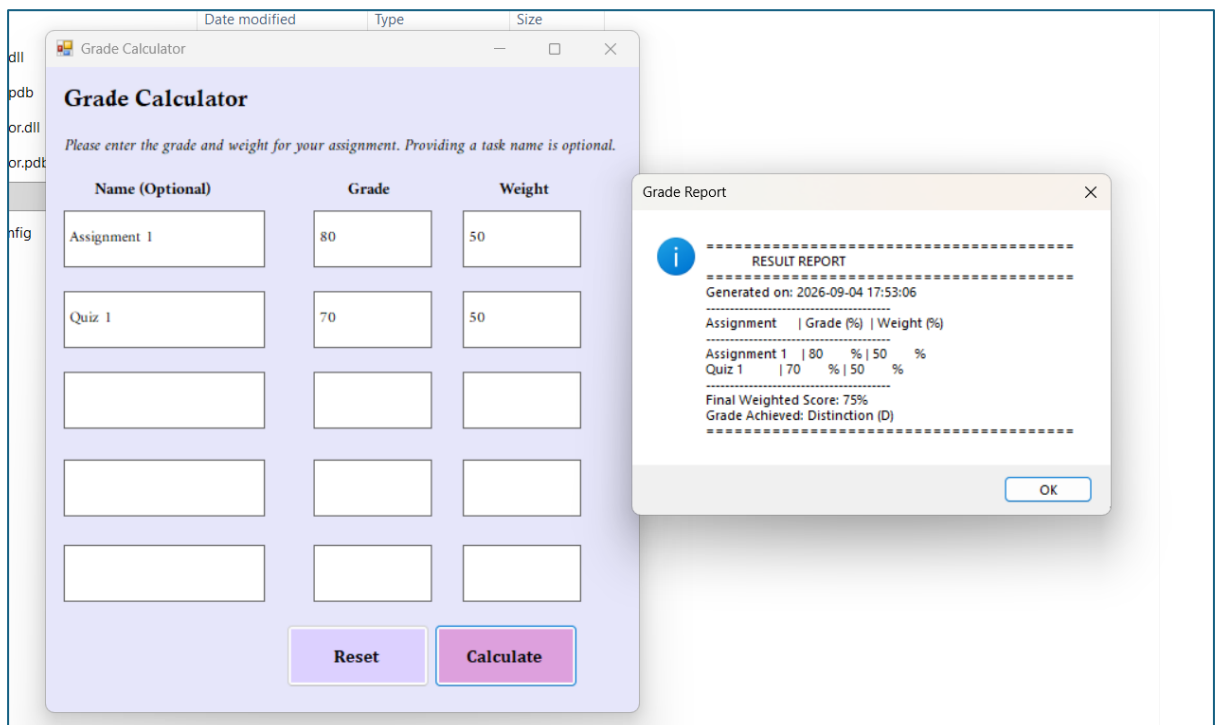


Figure 39: Screenshot of Grade Calculator application and the generated results dialog box.

7. Verification of Persistent Local Archiving

To verify that ResultGenerator.dll successfully performed its local file-handling operations, the computer's active Documents folder was inspected. The directory was verified to contain a new text file named Weighted_Grade_Report.txt. Opening the file in Notepad confirmed that the library successfully compiled and saved the identical detailed transcript.

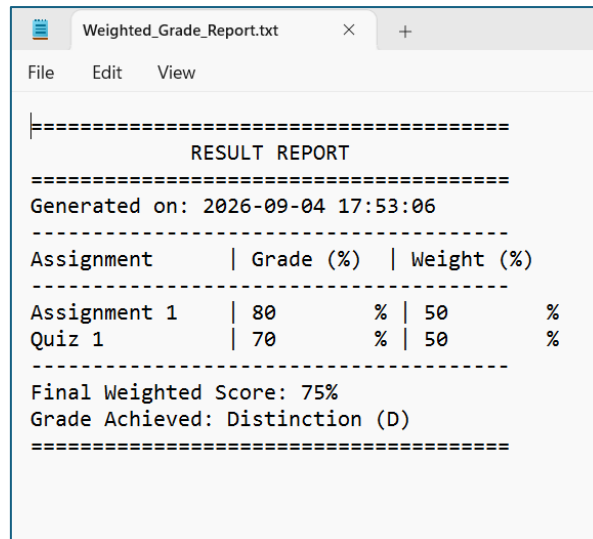


Figure 40: Screenshot of Weighted_Grade_Report

5.0 Task 1.4 - High Distinction

The objective of Task 1.4 is to transition the desktop application into a MSIX package format compatible with modern Windows operating systems and the Microsoft Store.

Because publishing a live application to the public Microsoft Store requires a paid developer account registration and an administrative review process that takes several days, this task was completed by successfully compiling and verifying a local sideloading package and providing a detailed technical blueprint for public Store publication.

5.1 Window Application Packaging Project Setup

A Window Application Packaging Project named WinUniversalPackage was added to the Visual Studio solution to package the Grade Calculator and its DLL dependencies. The packaging process was configured using Visual Studio's packaging wizard with the following specific configuration steps:

1. Create App Packages Wizard

In Visual Studio's Solution Explorer, the packaging project WinUniversalPackage was selected. Right-clicking the project and navigating to Publish > Create App Packages initialized the deployment wizard.

2. Distribution Selection

In the initial step of the wizard, Sideloaded was selected as the distribution method. Sideloaded allows the signed package bundle to be distributed and installed directly on local target machines for testing and academic verification, bypassing the public Microsoft Store submission for this run.

3. Selecting the Custom Digital Certificate

To satisfy Windows security constraints, the package had to be signed. Under the signing settings, the custom test certificate was selected to digitally sign the package.

4. Target Architecture Selection

In the architecture configuration step, Neutral (AnyCPU) was selected as the target architecture. Selecting a neutral build configuration ensures that the compiled .msixbundle remains compatible with all standard CPU architectures without requiring separate architecture-specific compilations.

5. Package Compilation, Installation, and Launch

Once the wizard steps were completed, the compiler generated the signed package bundle successfully. With this setup finalized, the application was successfully installed on the local operating system and launched directly as a modern Windows app, verifying that all system configurations, digital signatures, and dynamic-link libraries function cleanly together.

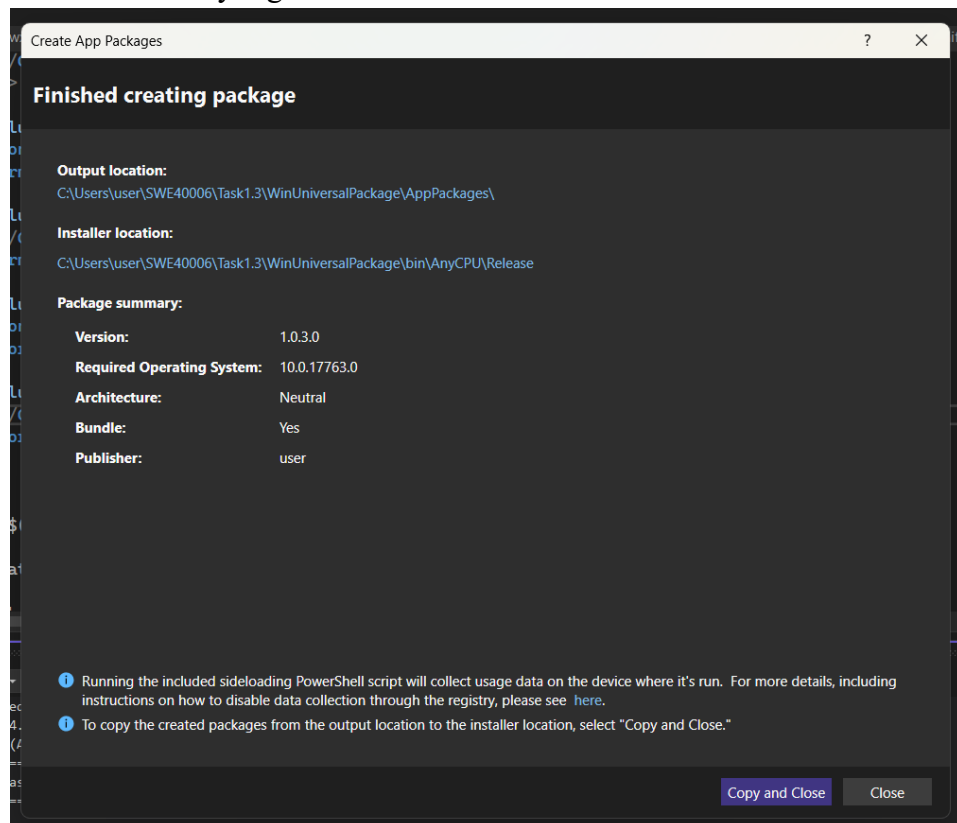


Figure 41: The Visual Studio "Create App Packages" wizard confirming successful compilation of your modern package bundle

5.2 Package Installation and Troubleshooting

During the packaging and installation process, two issues were identified and resolved.

1. Missing .cer File

- Problem

During the initial installation attempt, Windows displayed an error saying that the package or bundle is not digitally signed or its signature is corrupted. The package was generated without an associated signing certificate. As a result, Windows could not verify the authenticity of the MSIX/APPX package and prevented it from being installed.

- **Resolution**

The Package.appxmanifest file was opened in Visual Studio, and the Packaging section was selected. Under Choose Certificate, the Create Test Certificate option was used to generate a self-signed development certificate with the publisher's name Chai Yen Thung. The application package was then rebuilt using the new certificate. After rebuilding, the signed WinUniversalPackage_1.0.0.0_AnyCPU package and the required installation files were successfully generated. This resolved the signature validation issue and allowed the package to proceed to installation.

Use this page to set the properties that identify and describe your package when it is deployed.

Package name: ff4d3c5d-6ac3-4cb1-b878-1cc801081c3a

Package display name: WinUniversalPackage

Version: Major: 1 Minor: 0 Build: 3 [More information](#)

Publisher: CN=Chai Yen Thung [Choose Certificate...](#)

Publisher display name: user

Package family name: ff4d3c5d-6ac3-4cb1-b878-1cc801081c3a_9dtgkzz0bdtgr

Figure 42: The packaging manifest settings editor showing self-signed certificates

2. PowerShell Execution Policy Block

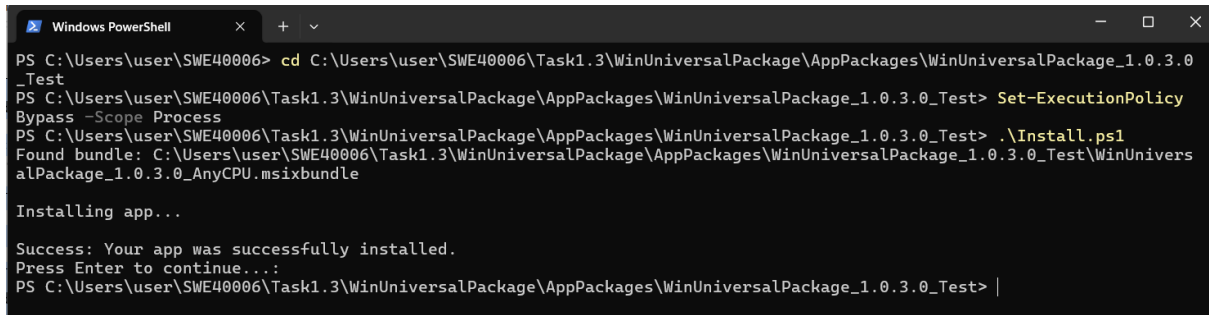
- **Problem**

When the generated Install PowerShell script was selected using Run with PowerShell, the PowerShell window opened briefly and then closed without completing the installation. The script was being prevented from executing because of the system's PowerShell execution policy.

- **Resolution**

Windows PowerShell was opened manually with administrator privileges. Instead of permanently changing the system-wide execution policy, the restriction was temporarily bypassed only for the current PowerShell process using `Set-ExecutionPolicy Bypass -Scope Process`. The terminal was then navigated to the generated package directory, and the installation script was executed manually using: `.\Install.ps1`

This allowed the installation script to execute successfully and deploy the packaged application to the local machine. Using `-Scope Process` ensured that the execution-policy change applied only to the current PowerShell session rather than permanently modifying the system's security configuration.



```
Windows PowerShell
PS C:\Users\user\SWE40006> cd C:\Users\user\SWE40006\Task1.3\WinUniversalPackage\AppPackages\WinUniversalPackage_1.0.3.0_Test
PS C:\Users\user\SWE40006\Task1.3\WinUniversalPackage\AppPackages\WinUniversalPackage_1.0.3.0_Test> Set-ExecutionPolicy Bypass -Scope Process
PS C:\Users\user\SWE40006\Task1.3\WinUniversalPackage\AppPackages\WinUniversalPackage_1.0.3.0_Test> .\Install.ps1
Found bundle: C:\Users\user\SWE40006\Task1.3\WinUniversalPackage\AppPackages\WinUniversalPackage_1.0.3.0_Test\WinUniversalPackage_1.0.3.0_AnyCPU.msixbundle
Installing app...
Success: Your app was successfully installed.
Press Enter to continue...
PS C:\Users\user\SWE40006\Task1.3\WinUniversalPackage\AppPackages\WinUniversalPackage_1.0.3.0_Test> |
```

Figure 43: Elevated PowerShell console executing the install script

5.3 Detailed Microsoft Store Deployment Blueprint

To take the compiled `.msixbundle` and make it accessible on the public Microsoft Store for public download, a developer must complete the following administrative and technical steps.

1. Partner Center Account Registration

The developer registers a verified Windows Developer Account with the Microsoft Partner Center. A one-time registration fee around 19 USD is paid to verify identity.

2. Product Name Reservation

A unique product name is reserved within the Store dashboard.

3. Visual Studio Store Association

Inside Visual Studio, the developer right-clicks the packaging project, selects `Publish > Associate App with the Store`, and logs in. This automatically updates the local package manifest with the official Store identity and replaces the local test certificate with Microsoft's signed publisher key.

4. Generating the Store Bundle

The developer runs the Create App Packages wizard, selecting Microsoft Store instead of sideloading. This compiles the release files and generates a single `.msixupload` package.

5. Submission Configuration

Under the reserved app dashboard in Partner Center, the developer configures:

- **Pricing and Availability:** Free or paid model, and targeted geographic regions.
- **Properties:** Defining app category (Utilities/Educational).
- **Age Ratings:** Completing the global IARC questionnaire to determine regional content ratings.
- **Store Listings:** Adding app descriptions, search terms, and screenshots.

6. Package Upload and Certification

The .msixupload package is uploaded. The developer submits the app for Certification. Microsoft runs automated static analysis scans to ensure code safety, followed by manual validation by a Microsoft tester. Once certified, the app becomes searchable and downloadable on the public Microsoft Store.

5.4 Deployment Outcomes

To verify the successful creation, digital signing, and local installation of the modern package, the process was executed and documented in the following steps.

1. Verification of the Signed Output Directory

Once the package creation wizard was completed, the compiler generated the package files inside the output folder. This folder confirms the presence of the completed WinUniversalPackage_1.0.3.0_AnyCPU.msixbundle package, the automated Install.ps1 PowerShell script, and the newly generated WinUniversalPackage_1.0.3.0_AnyCPU.cer security certificate.

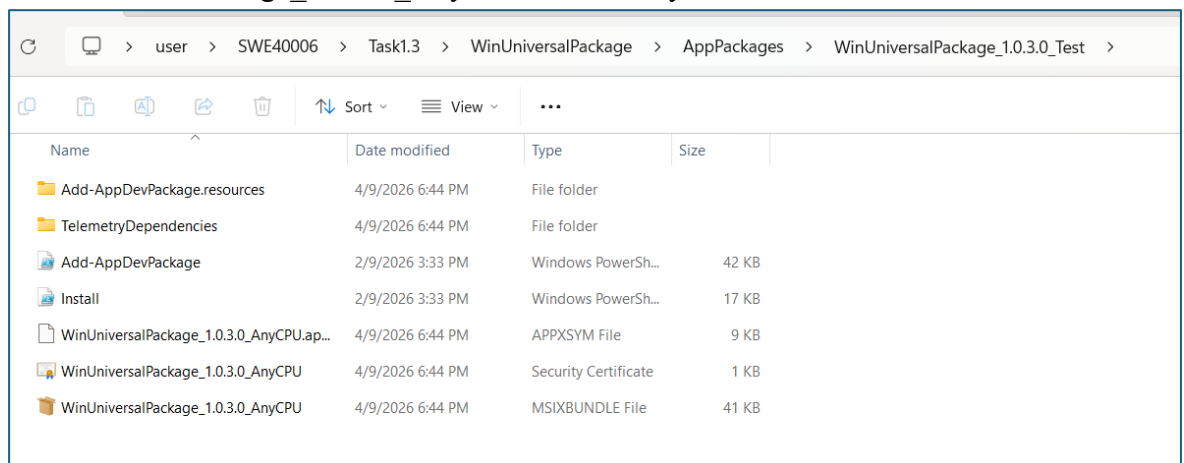


Figure 44: Screenshot of the signed package output directory, verifying the presence of the .msixbundle and the generated .cer digital security certificate.

2. Verification of Package Integrity and Digital Signature

Double-clicking the .msixbundle file launched the native Windows App Installer. Because the package was successfully signed using the custom certificate created under the publisher identity Chai Yen Thung, Windows recognized the bundle as secure and trusted, displaying the application information and version 1.0.3.0.

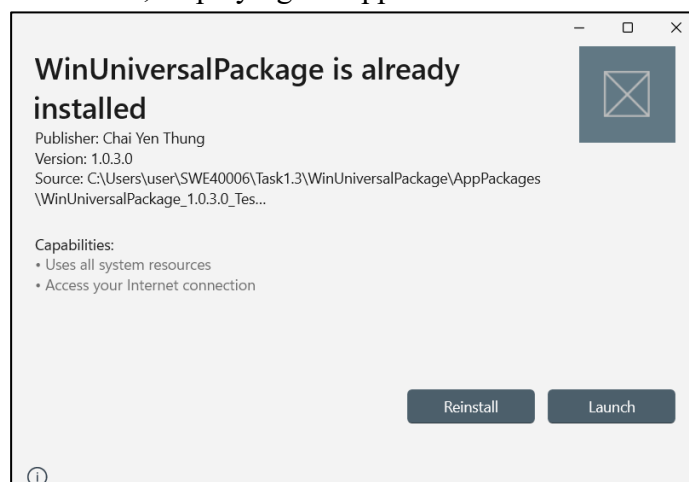


Figure 45: Screenshot of the Windows App Installer interface, verifying that the package is digitally signed by publisher and trusted by the operating system

3. Verification of Sideloaded Installation and Launch

Running the installation script successfully deployed the application container. Clicking Launch directly from the installer window confirmed that the modern containerized application executes successfully, launching the Grade Calculator interface on the desktop.

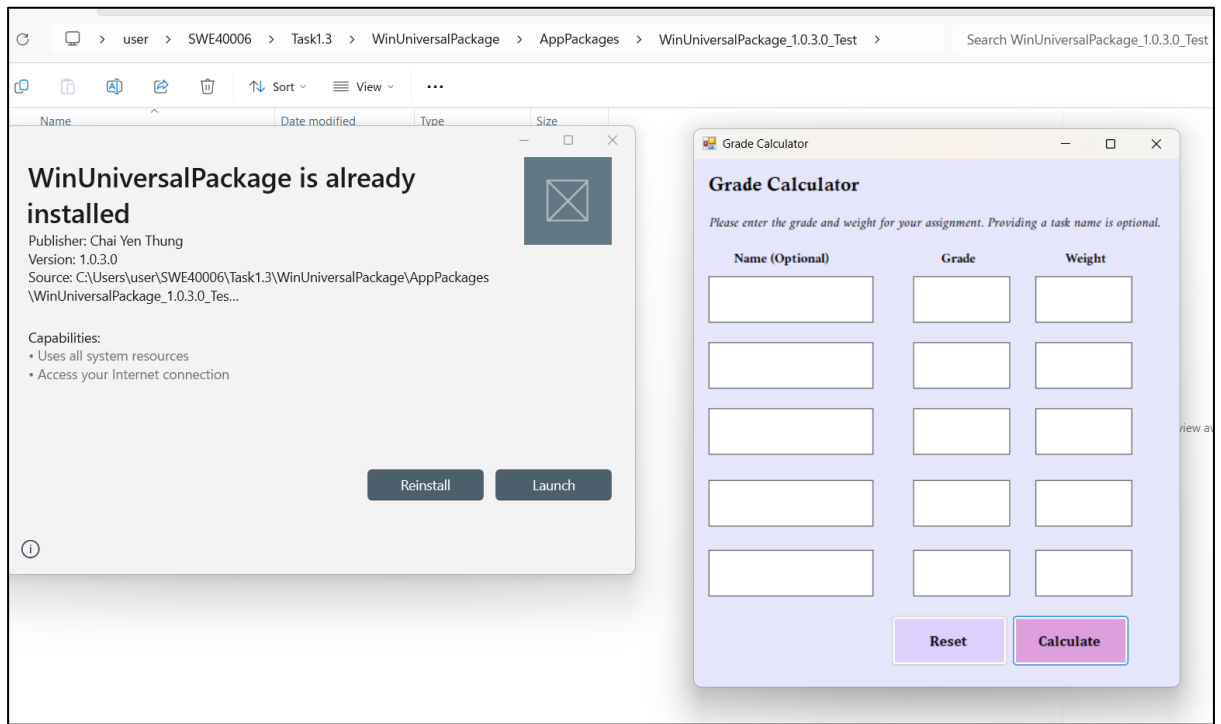


Figure 46: Screenshot of the deployed modern package launching the Grade Calculator on the desktop side-by-side with the installer interface

4. Verification of App Registration

To confirm that Windows registered the containerized package, the system's "Installed apps" settings panel was checked. The search confirmed that WinUniversalPackage was successfully registered on the system under the user account.

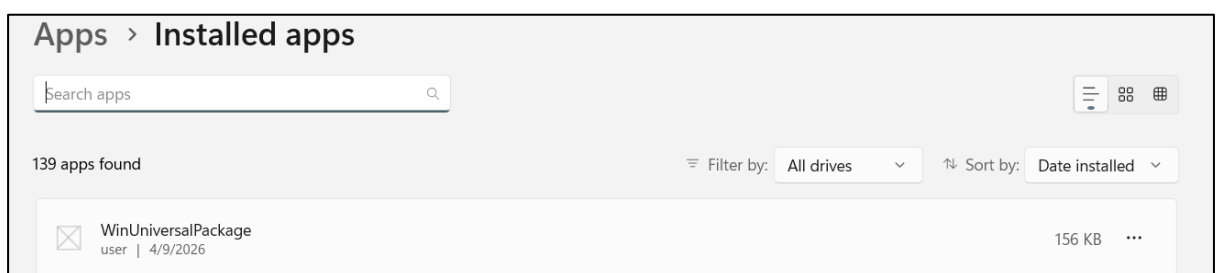


Figure 47: Screenshot of the Windows "Installed apps" settings panel confirming the containerized modern package is successfully registered in the operating system database.